

WEEK 3 MODULE

1. Merge Sort:

Definition: Merge Sort is a Sorting algorithm that divides an array into smaller parts, sorts each part separately, and then merges them to obtain a completely sorted array.

Algorithm: (Step by Step)

Step 1: Create two temporary arrays:

- Left array = $A[\text{left} \dots \text{mid}]$
- Right array = $A[\text{mid} + 1 \dots \text{right}]$

Step 2: Copy the element into the temporary arrays

Step 3: Compare the first element of both arrays

Step 4: Copy the smaller element into the original array

Step 5: Move to the next element of the array element was copied

Step 6: Repeat step 4-6 until one temporary array empty

Step 7: Copy all remaining element of left array

Step 8: Copy all remaining element of Right array

Step 9: Stop.

Step 10:

Time Complexity: $O(n \log n)$

Problem 1: (Code of Merge Sort)

```
#include <iostream>
```

```
using namespace std
```

```
void merge(int arr[], int l, int m, int r) {
```

```
    int n1 = m - l + 1;
```

```
    int n2 = r - m;
```

```
    int L[n1], R[n2];
```

```
    for(int i = 0; i < n1; i++) {
```

```
        L[i] = arr[l + i];
```

```
    }
```

```
    for(int j = 0; j < n2; j++) {
```

```
        R[j] = arr[m + 1 + j];
```

```
    }
```

```
    int i = 0, j = 0, k = l;
```

```
    while (i < n1 && j < n2) {
```

```
        if (L[i] <= R[j]) {
```

```
            arr[k++] = L[i++];
```

```
        } else {
```

```
            arr[k++] = R[j++];
```

```
        }
```

```
    }
```

```
    while (i < n1) {
```

```
        arr[k++] = L[i++];
```

```
    }
```

```
    while (j < n2) {
```

```
        arr[k++] = R[j++];
```

```
    }
```



```

void mergesort(int arr[], int l, int r) {
    if (l < r) {
        int m = (l + r) / 2;
        mergesort(arr, l, m);
        mergesort(arr, m + 1, r);
        merge(arr, l, m, r);
    }
}

```

```

int main() {
    int n;
    cin >> n;
    int arr[n];
    for (int i = 0; i < n; i++) {
        cin >> arr[i];
    }
    mergeSort(arr, 0, n - 1);
    for (int i = 0; i < n; i++) {
        cout << arr[i] << " ";
    }
    return 0;
}

```

Welcome

practice.cpp ×

Users > ankitkumarsingh > practice.cpp > main()

```
118 void mergeSort(int arr[], int left, int right)
119 {
120     if (left < right)
121     {
122         int mid = left + (right - left) / 2;
123
124         mergeSort(arr, left, mid);
125         mergeSort(arr, mid + 1, right);
126
127         merge(arr, left, mid, right);
128     }
129 }
130
131 int main()
132 {
133     int n;
134     cin >> n;
135
136     int arr[n];
137
138     for (int i = 0; i < n; i++)
139         cin >> arr[i];
140
141     mergeSort(arr, 0, n - 1);
142
143     for (int i = 0; i < n; i++)
144         cout << arr[i] << " ";
145
146     return 0;
147 }
```

PROBLEMS

OUTPUT

DEBUG CONSOLE

TERMINAL

PORTS

```
• ankitkumarsingh@ankits-MacBook-Air ~ % /Users/ankitkumarsingh/practice
6
450 1200 800 650 300 1500
300 450 650 800 1200 1500
ankitkumarsingh@ankits-MacBook-Air ~ %
```



Problem 2:

```
#include <iostream>
```

```
using namespace std;
```

```
void merge(int arr[], int left, int mid, int right) {
```

```
    int n1 = mid - left + 1;
```

```
    int n2 = right - mid;
```

```
    int L[n1], R[n2];
```

```
    for (int i = 0; i < n1; i++) {
```

```
        L[i] = arr[left + i];
```

```
    }
```

```
    for (int j = 0; j < n2; j++) {
```

```
        R[j] = arr[mid + 1 + j];
```

```
    }
```

```
    int i = 0, j = 0, k = left;
```

```
    while (i < n1 && j < n2) {
```

```
        if (L[i] <= R[j]) {
```

```
            arr[k] = L[i];
```

```
            i++;
```

```
        } else {
```

```
            arr[k] = R[j];
```

```
            j++;
```

```
        }
```

```
        k++;
```

```
    }
```

```
    while (i < n1) {
```

```
        arr[k] = L[i];
```

```
        i++;
```

```
        k++;
```

```
    }
```



```

while(j < n/2) {
    arr[k] = arr[j];
    j++;
    k++;
}

```

```

}

```

```

void mergeSort(int arr[], int left, int right) {
    if(left < right) {
        int mid = left + (right - left) / 2;
        mergeSort(arr, left, mid);
        mergeSort(arr, mid+1, right);
        merge(arr, left, mid, right);
    }
}

```

```

}

```

```

int main() {
    int n;
    cin >> n;
    int arr[n];
    for(int i = 0; i < n; i++) {
        cin >> arr[i];
    }
}

```

```

mergeSort(arr, 0, n-1);

```

// Sorted array

```

for(int i = 0; i < n; i++) {
    cout << arr[i] << " ";
}

```

```

}

```

```

cout << endl;

```

// Median

```
int median = arr[n/2];  
cout << "median" << median << endl;
```

// Count greater than median

```
int count = 0;  
for(int i = 0; i < n; i++) {  
    if (arr[i] > median) {  
        count++;  
    }  
}  
cout << "Order above Median" << count << endl;
```

// Difference

```
int difference = arr[n-1] - arr[0];  
cout << "Difference:" << difference << endl;  
return 0;
```

Welcome

practice.cpp X

Users > ankitkumarsingh > practice.cpp > main()

```
137     for (int i = 0; i < n; i++)
138         cin >> arr[i];
139
140     mergeSort(arr, 0, n - 1);
141
142     // Sorted array
143     for (int i = 0; i < n; i++)
144         cout << arr[i] << " ";
145     cout << endl;
146
147     // Median
148     int median = arr[n / 2];
149     cout << "Median: " << median << endl;
150
151     // Count greater than median
152     int count = 0;
153     for (int i = 0; i < n; i++)
154     {
155         if (arr[i] > median)
156             count++;
157     }
158
159     cout << "Orders Above Median: " << count << endl;
160
161     // Difference
162     int difference = arr[n - 1] - arr[0];
163     cout << "Difference: " << difference << endl;
164
165     return 0;
166 }
```

PROBLEMS

OUTPUT

DEBUG CONSOLE

TERMINAL

PORTS

```
● ankitkumarsingh@ankits-MacBook-Air ~ % /Users/ankitkumarsingh/practice
7
12 25 18 30 15 22 40
12 15 18 22 25 30 40
Median: 22
Orders Above Median: 3
Difference: 28
ankitkumarsingh@ankits-MacBook-Air ~ %
```


2. Quick Sort :

Definition: Quick sort is a divide-and-conquer sorting algorithm. It works by selecting one element as a pivot placing all smaller elements to the left of the pivot and all larger elements to the right and then recursively sorting the left and right parts.

Algorithm: (Step by step)

Step 1: Start

Step 2: choose a pivot element from the array.

(commonly) the last first or middle element

Step 3: Rearrange the array so that.

- All elements smaller than the pivot come before it.

- All elements greater than the pivot come after it.

Step 4: The pivot is now in its correct sorted position

Step 5: Apply quick sort recursively to the left subarray.

Step 6: Apply quick sort recursively to the right subarray.

Step 7: Repeat until each subarray has 0 or 1 element.

Step 8: Stop.

- Time complexity: $O(n^2)$

Problem 1: (Quick Sort)

```
#include <iostream>
using namespace std;
int partition(int arr[], int low, int high) {
    int pivot = arr[high];
    int i = low - 1;
    for(int j = low; j < high; j++) {
        if (arr[j] < pivot) {
            i++;
            swap(arr[i], arr[j]);
        }
    }
    swap(arr[i+1], arr[high]);
    return i+1;
}

void quicksort(int arr[], int low, int high) {
    if (low < high) {
        int pi = partition(arr, low, high);
        quicksort(arr, low, pi-1);
        quicksort(arr, pi+1, high);
    }
}

int main() {
    int N;
    cin >> N;
    int arr[N];
    for(int i = 0; i < N; i++) {
        cin >> arr[i];
    }
}
```

```
quickSort(arr, 0, N-1);
```

```
for (int i = 0; i < N; i++) {  
    cout << arr[i] << " ";  
}
```

```
return 0;
```

```
}
```


Welcome

practice.cpp ×

Users > ankitkumarsingh > practice.cpp > main()

```
172 int partition(int arr[], int low, int high)
189
190 void quickSort(int arr[], int low, int high)
191 {
192     if (low < high)
193     {
194         int pi = partition(arr, low, high);
195
196         quickSort(arr, low, pi - 1);
197         quickSort(arr, pi + 1, high);
198     }
199 }
200
201 int main()
202 {
203     int N;
204     cin >> N;
205
206     int arr[N];
207
208     for (int i = 0; i < N; i++)
209         cin >> arr[i];
210
211     quickSort(arr, 0, N - 1);
212
213     for (int i = 0; i < N; i++)
214         cout << arr[i] << " ";
215
216     return 0;
217 }
```

PROBLEMS

OUTPUT

DEBUG CONSOLE

TERMINAL

PORTS

```
● ankitkumarsingh@ankits-MacBook-Air ~ % /Users/ankitkumarsingh/practice
6
850 1200 950 600 1500 1000
600 850 950 1000 1200 1500
❖ ankitkumarsingh@ankits-MacBook-Air ~ %
```

Problem 2: (Quick Sort)

```
#include <iostream>
```

```
using namespace std;
```

```
int partition(int arr[], int low, int high) {
    int pivot = arr[high]
    int i = low - 1;
```

```
    for (int j = low; j < high; j++) {
```

```
        if (arr[j] > pivot) {
```

```
            i++;
```

```
            swap(arr[i], arr[j]);
        }
```

```
    }
```

```
    swap(arr[i+1], arr[high]);
```

```
    return i+1;
```

```
}
```

```
void quicksort(int arr[], int low, int high) {
```

```
    if (low < high) {
```

```
        int p = partition(arr, low, high);
```

```
        quicksort(arr, low, p-1);
```

```
    }
```

```
}
```

```
int main() {
```

```
    int N;
```

```
    cin >> N;
```

```
    int arr[N];
```

```
    for (int i = 0; i < N; i++) {
```

```
        cin >> arr[i];
```

```
    }
```



```
quicksort(arr, 0, N-1);  
cout << "Sorted Value :";  
for (int i = 0; i < N; i++) {  
    cout << arr[i] << " ";  
}  
cout << endl;
```

```
cout << "Top 5";  
long long topSum = 0;
```

```
for (int i = 0; i < 5 && i < N; i++) {  
    cout << arr[i] << " ";  
    topSum += arr[i];  
}  
cout << endl;
```

```
double topAverage = (double) topSum / 5;  
cout << "Average of Top 5" << topAverage << endl;
```

```
long long total = 0;  
for (int i = 0; i < N; i++) {  
    total += arr[i];  
}
```

```
double overallAverage = (double) total / N;
```

```
int count = 0;
```

for (int i = 0; i < N; i++) {
 if (arr[i] > overall Average) {
 Count++;
 }
}

}

cout << "Value Above Overall Average" << Count;

return 0;

}



Welcome

practice.cpp ×

Users > ankitkumarsingh > practice.cpp > main()



```
221     long long topSum = 0;
222
223     for (int i = 0; i < 5 && i < N; i++)
224     {
225         cout << arr[i] << " ";
226         topSum += arr[i];
227     }
228
229     cout << endl;
230
231     double topAverage = (double)topSum / 5;
232     cout << "Average of Top 5: " << topAverage << endl;
233
234     long long total = 0;
235     for (int i = 0; i < N; i++)
236         total += arr[i];
237
238     double overallAverage = (double)total / N;
239
240     int count = 0;
241     for (int i = 0; i < N; i++)
242     {
243         if (arr[i] > overallAverage)
244             count++;
245     }
246
247     cout << "Values Above Overall Average: " << count;
248
249     return 0;
250 }
```

PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS

```
● ankitkumarsingh@ankits-MacBook-Air ~ % /Users/ankitkumarsingh/practice
8
12 7 15 9 20 5 10 8
Sorted Values: 20 15 12 10 9 8 7 5
Top 5: 20 15 12 10 9
Average of Top 5: 13.2
Values Above Overall Average: 3
ankitkumarsingh@ankits-MacBook-Air ~ %
```



3. Heap Sort:

Definition: Heap Sort is a comparison-based sorting algorithm that uses a Binary Heap data structure to sort elements in ascending or descending order.

Algorithm: (Step-by-Step)

Step 1: Store all elements in an array.

Step 2: Build a max Heap. The parents given array.

- In a max Heap, the parents node is always greater than or equal to its children.

Step 3: Swap the root element (largest element) with the last element of the heap.

Step 4: Reduce the heap size by 1 (ignore the last sorted element).

Step 5: Apply Heapify to the root to restore the max Heap property.

Date / /

Step 6 : Repeat Step 3-5 until only one element remains.

Step 7 : The array is now sorted in ascending order.

Problem 1: (Heap Sort)

```
#include <iostream>
using namespace std;
```

```
void heapify (int arr[], int n, int i) {
    int largest = i;
    int left = 2 * i + 1;
    int right = 2 * i + 2;

    if (right < n && arr[left] > arr[largest]) {
        largest = left;
    }
    if (right < n && arr[right] > arr[largest]) {
        largest = right;
    }
    if (largest != i) {
        swap(arr[i], arr[largest]);
        heapify(arr, n, largest);
    }
}
```

```
void heapsort (int arr[], int n) {
    for (int i = n / 2 - 1; i >= 0; i--) {
        heapify(arr, n, i);
    }
    for (int i = n - 1; i > 0; i--) {
        swap(arr[0], arr[i]);
        heapify(arr, i, 0);
    }
}
```



```
int main() {
```

```
    int N;
```

```
    cin >> N;
```

```
    int arr[N];
```

```
    for(int i = 0; i < N; i++) {
```

```
        cin >> arr[i];
```

```
    }
```

```
    heapsort(arr, N)
```

```
    for(int i = 0; i < N; i++) {
```

```
        cout << arr[i] << " ";
```

```
    }
```

```
    return 0;
```

```
}
```

Welcome

dsa notes.cpp

Users > ankitkumarsingh > dsa notes.cpp > main()

```

500     for (int i = 0; i < n; i++)
501     {
502         cin >> arr[i];
503         sum += arr[i];
504     }
505
506     heapSort(arr, n);
507
508     cout << "Sorted Response Times:\n";
509     for (int i = 0; i < n; i++)
510         cout << arr[i] << " ";
511
512     double avg = sum / n;
513
514     int count = 0;
515     for (int i = 0; i < n; i++)
516     {
517         if (arr[i] < avg)
518             count++;
519     }
520
521     double percentage = (count * 100.0) / n;
522
523     cout << "\nFastest: " << arr[0];
524     cout << "\nSlowest: " << arr[n - 1];
525     cout << "\nAverage: " << fixed << setprecision(2) << avg;
526     cout << "\nCases Faster Than Average: " << count;
527     cout << "\nPercentage: " << fixed << setprecision(2) << percentage << "%";
528
529     return 0;
530 }
```

PROBLEMS

OUTPUT

DEBUG CONSOLE

TERMINAL

PORTS

```

● ankitkumarsingh@ankits-MacBook-Air ~ % "/Users/ankitkumarsingh/dsa notes"
8
12 7 15 9 20 5 15 8
Sorted Response Times:
5 7 8 9 12 15 15 20
Fastest: 5
Slowest: 20
Average: 11.38
Cases Faster Than Average: 4
Percentage: 50.00%
○ ankitkumarsingh@ankits-MacBook-Air ~ %
```


Problem 2: (Heap Sort);

```
#include <iostream>
```

```
#include <iomanip>
```

```
using namespace std;
```

```
void heapify (int arr[], int n, int i) {
```

```
    int largest = i;
```

```
    int left = 2 * i + 1;
```

```
    int right = 2 * i + 2;
```

```
    if (left < n && arr[left] > arr[largest])
```

```
        largest = left;
```

```
    if (right < n && arr[right] > arr[largest])
```

```
        largest = right;
```

```
    if (largest != i) {
```

```
        swap (arr[i], arr[largest]);
```

```
        heapify (arr, n, largest);
```

```
    }
```

```
// Build max Heap
```

```
for (int i = n/2 - 1; i >= 0; i--)
```

```
    heapify (arr, n, i);
```

```
// Heap Sort
```

```
for (int i = n - 1; i > 0; i--)
```

{

```
    swap(arr[0], arr[i]);  
    heapify(arr, i, 0);  
}
```

}

in main()

{

```
    int n;  
    cin >> n;
```

```
    int arr[n];
```

```
    for (int i = 0; i < n; i++)  
        cin >> arr[i];
```

```
    heapSort(arr, n);
```

```
    cout << "Sorted Response times: ";  
    for (int i = 0; i < n; i++)  
        cout << arr[i] << " ";  
    cout << endl;
```

```
    int fastest = arr[0];
```

```
    int slowest = arr[n-1];
```

```
    double sum = 0;
```

```
    for (int i = 0; i < n; i++)  
        sum += arr[i];
```


double average = Sum / n;

int count = 0;

for (int i = 0; i < n; i++)

{

if (arr[i] < average)
count ++;

}

double percentage = (count * 100.0) / n;

cout << "Fastest: " << fastest << endl;

cout << "Slowest: " << slowest << endl;

cout << fixed << setprecision(2);

cout << "Average: " << average << endl;

cout << "cases Faster Than Average: " <<

cout << "percentage: " << percentage << "

return 0;

}

Welcome

dsa notes.cpp ×

Users > ankitkumarsingh > dsa notes.cpp > main()

```
479 void heapSort(int arr[], int n)
482     for (int i = n / 2 - 1; i >= 0; i--)
483         heapify(arr, n, i);
484
485     // Extract elements one by one
486     for (int i = n - 1; i > 0; i--)
487     {
488         swap(arr[0], arr[i]);
489         heapify(arr, i, 0);
490     }
491 }
492
493 int main()
494 {
495     int N;
496     cin >> N;
497
498     int arr[N];
499
500     for (int i = 0; i < N; i++)
501         cin >> arr[i];
502
503     heapSort(arr, N);
504
505     // Print Sorted Array
506     for (int i = 0; i < N; i++)
507         cout << arr[i] << " ";
508
509     return 0;
510 }
```

PROBLEMS

OUTPUT

DEBUG CONSOLE

TERMINAL

PORTS

```
● ankitkumarsingh@ankits-MacBook-Air ~ % "/Users/ankitkumarsingh/dsa notes"
6
850 1200 950 600 1500 1000
600 850 950 1000 1200 1500
○ ankitkumarsingh@ankits-MacBook-Air ~ %
```